

composer create-project laravel/laravel todo

php artisan serve – uruchamia lokalny serwer testowy.

php artisan make:... – służy do generowania plików (np. modeli, kontrolerów, widoków).

php artisan migrate – tworzy tabele w bazie danych na podstawie plików migracji.

php artisan route:list – wyświetla listę wszystkich dostępnych tras w aplikacji.

Routing (Trasy)

Zwykła trasa z funkcją: Route::get('/about', function () { return 'Strona'; });

Trasa do kontrolera: Route::get('/posts', [PostController::class, 'index']); (URL -> Controller -> metoda).

Trasa z parametrami: Route::get('/users/{id}', ...).

Nazwane trasy: Route::get('/about', ...)->name('about'); Pozwalają później łatwo wywołać adres URL za pomocą polecenia route('about').

Grupowanie (np. z prefiksem admin): Route::prefix('admin')->group(function() { ... });

Fallback (trasa awaryjna 404): Route::fallback(function () { return 'Nie znaleziono'; }); (Musi być na samym końcu pliku!).

Route::resource('tasks', TaskController::class); - Polecenie wygeneruje Controller z siedmioma predefiniowanymi funkcjami.

Kontrolery i Walidacja

Tworzenie kontrolera: php artisan make:controller CarController.

Single Action Controller: Kontroler posiadający tylko jedną metodę __invoke(), która odpowiada za jedną konkretną akcję.

Resource Controller: (CRUD): index(), create(), store(), show(), edit(), update(), destroy(). Wywołanie to php artisan make:controller ProductController --resource.

Walidacja w kontrolerze:

```
$validated = $request->validate([
    'name' => 'required|string|max:255',
    'email' => 'required|email',
    'age' => 'required|integer|min:18',
    'password' => 'required|confirmed'
]);
```

Baza danych – Migracje, Factory i Seeder

Migracje: Pozwalają na pisanie struktury bazy w kodzie PHP (zamiast używać SQL) i wersjonowanie tych zmian.

Utworzenie modelu wraz z kompletem plików (migracja, factory, seeder, kontroler): php artisan make:model Person -mfsc.

Przykładowa migracja: \$table->string('name'); \$table->boolean('completed')->default(false); \$table->foreignId('user_id')->constrained();

php artisan migrate

Factory: Generuje przykładowe, testowe dane. Korzysta z biblioteki **Faker**, np. fake()->name(), fake()->email(), fake()->sentence()

Seeder: Narzędzie do uzupełniania bazy zdefiniowanymi lub testowymi danymi. Wywoływane poleceniem php artisan db:seed.

Całkowite usunięcie i odtworzenie bazy ze wstawieniem danych: php artisan migrate:fresh --seed

1. Inicjalizacja i generowanie plików (styl rest api)

```
// 1. Tworzymy nowy projekt
composer create-project laravel/laravel todo-api
// 2. Wchodzimy do folderu i instalujemy obsługę API (Laravel 11+)
cd todo-api
php artisan install:api
// 3. Generujemy Model wraz z Migracją, Factory, Seederem i Kontrolerem
php artisan make:model Task -mfsc
// 4. Generujemy Resource (do formatowania JSON) i Policy (do autoryzacji)
php artisan make:resource TaskResource
php artisan make:policy TaskPolicy --model=Task
// 5. Generujemy kontroler do logowania
php artisan make:controller Api/AuthController
```

2. Baza danych (Migracje, Factory, Seeder)

Definiujemy, jak wyglądają dane i tworzymy testowe wpisy.

Migracja:

```
public function up() {
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->boolean('completed')->default(false);
        $table->foreignId('user_id')->constrained()->cascadeOnDelete(); // Przypisanie do usera
        $table->timestamps();
    });
}
```

Model (Task.php):

```
protected $fillable = ['title', 'completed', 'user_id'];
```

Factory (TaskFactory.php):

```
public function definition() {
    return [
        'title' => fake()->sentence(),
        'completed' => fake()->boolean(),
    ];
}
```

Seeder (TaskSeeder.php i DatabaseSeeder.php):

```
public function run(): void
Task::factory()->count(10)->create(); // losowe dane
Task::create[
    "title=>"task1",
    "completed"=>"False",]);

php artisan migrate:fresh --seed
```

3. Autentykacja (Logowanie i Rejestracja):

W AuthController.php pozwalamy użytkownikowi wejść do systemu i dajemy mu "przepustkę" w postaci tokena

```
public function register(Request $request) {  
    // 1. Walidacja (name, email, password)  
    // 2. Utworzenie usera: User::create([...])  
    // 3. Wygenerowanie tokena:  
    $token = $user->createToken('api-token')->plainTextToken;  
    return response()->json(['token' => $token], 201);  
}
```

```
public function login(Request $request) {  
    // 1. Sprawdzenie poprawności danych  
    // 2. Zwrócenie tokena j.w.  
}
```

4. Formatowanie i Autoryzacja

Policy (TaskPolicy.php):

// Użytkownik może edytować/usunąć TYLKO swoje zadanie

```
public function update(User $user, Task $task) {  
    return $user->id === $task->user_id;  
}
```

Resource (TaskResource.php):

// Formatujemy to, co zobaczy frontend

```
public function toArray($request) {  
    return [  
        'id' => $this->id,  
        'title' => $this->title,  
        'completed' => $this->completed  
    ];  
}
```

5. Główna logika (TaskController)

```
class TaskController extends Controller {
```

// READ: Lista zadań zalogowanego usera

```
public function index() {  
    $tasks = Task::where('user_id', auth()->id()->get());  
    return TaskResource::collection($tasks);  
}
```

// CREATE: Nowe zadanie

```
public function store(Request $request) {  
    $request->validate(['title' => 'required']);  
    $task = Task::create([  
        'title' => $request->title,  
        'user_id' => auth()->id() // przypisujemy autora  
    ]);  
    return new TaskResource($task);  
}
```

```

    }
    // UPDATE: Edycja zadania
    public function update(Request $request, Task $task) {
        $this->authorize('update', $task); // Sprawdzenie w Policy
        $task->update($request->all());
        return new TaskResource($task);
    }
    // DELETE: Usuwanie
    public function destroy(Task $task) {
        $this->authorize('delete', $task); // Sprawdzenie w Policy
        $task->delete();
        return response()->json(['message' => 'Usunięto']);
    }
}

```

6. Routing (routes/api.php)

```

use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\TaskController;

// Ogólnodostępne
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

// Tylko dla zalogowanych (wymagają wystania Tokena)
Route::middleware('auth:sanctum')->group(function () {

    // Generuje gotowe trasy dla index, store, show, update, destroy
    Route::apiResource('tasks', TaskController::class);

});

```

7. Odpalenie

```

// Buduje tabele w bazie i od razu wrzuca do nich 10 testowych zadań
php artisan migrate:fresh --seed

// Odpala aplikację
php artisan serve

```

1. Inicjalizacja i generowanie plików

```
// 1. Tworzymy nowy projekt webowy (bez API)
composer create-project laravel/laravel todo-app
// 2. Wchodzimy do folderu
cd todo-app
// 3. Generujemy Model, Migrację i Kontroler
php artisan make:model Task -mfsc ((seeder i factory z API)
// 4. Tworzymy pliki widoków Blade (katalog tworzymy ręcznie lub komendą)
php artisan make:view tasks.index
php artisan make:view tasks.create
```

2. Baza danych (Migracje i Model, tak jak w API)

3. Routing (routes/web.php)

```
use App\Http\Controllers\TaskController;
// Generuje trasy webowe: index, create, store, show, edit, update, destroy
Route::resource('tasks', TaskController::class);
```

4. Główna logika (TaskController)

```
class TaskController extends Controller {
// READ: Wyświetla listę zadań
public function index() {
    $tasks = Task::all();
    return view('tasks.index', compact('tasks'));
}
// CREATE: Wyświetla pusty formularz
public function create() {
    return view('tasks.create');
}
// STORE: Zapisuje dane z formularza
public function store(Request $request) {
    $request->validate(['title' => 'required']);
    Task::create($request->all());
    return redirect()->route('tasks.index');
}
// UPDATE: Oznacza jako zrobione
public function update(Request $request, Task $task) {
    $task->update(['completed' => true]);
    return redirect()->route('tasks.index');
}
// DELETE: Usuwa zadanie
public function destroy(Task $task) {
    $task->delete();
    return redirect()->route('tasks.index');
}
}
```

5. Widoki (Blade) Widok listy zadań (resources/views/tasks/index.blade.php):

Widok formularza (resources/views/tasks/create.blade.php): || (tasks/create.blade.php), pamiętaj, żeby w środku tagu <form> bezwzględnie dopisać dyrektywę @csrf

6. Odpalenie

```
php artisan migrate:fresh --seed // Buduje tabele w bazie i wrzuca 10 losowych zadań z seedera
php artisan serve // Odpala aplikację php artisan serve
```